



The Polsia Builder Playbook

How to Run the Builder Like a Pro, Ship Faster, and Stop Burning Credits

SkillForge AI

By Bill Morton

Second edition — September 2026



Introduction: Who This Is For

If you build websites on Polsia, you already know the builder. You describe what you want in plain language, it goes off and does the work, and it bills you in credits — one task, one credit.

What nobody tells you is how much of that bill is optional.

I run five e-commerce sites on Polsia. Over the last few months I've pushed hundreds of builder tasks through the system: publishing 47 products in a single shot, swapping buyer files across entire catalogs, fixing pricing, installing PDFs, wiring up pages, repairing what broke. I've learned — mostly by overpaying first — exactly where the credits leak and how to plug every hole.

This playbook is the whole system. The exact sentences to paste into your briefs. The traps the builder sets (it sets them, trust me). The quality gate that keeps you from shipping garbage. The file-handoff plumbing that nobody documents. And the copy-paste templates I use every single time.

No theory. Everything in here is something I did, paid for, and verified on the live page. When I give you a number, it's a number from my own receipts.

What this book is not

This is not a design guide. It won't teach you layout, branding, or what makes a good product page. It assumes you know what you want built.

It's also not a love letter to the builder. The builder is a tool — a fast, powerful, occasionally sneaky tool. This book is about the transaction between you and that tool: how to ask, how to check, and how to never pay twice for the same work.

How to use this book

Read it straight through once — it's short on purpose. Then keep it open next to you when you write briefs. Chapters 2, 3, and 4 are the money chapters; the appendix templates are the ones you'll copy weekly.

In this book:

- How the builder actually bills you — and the one sentence that cuts the bill
- The exact bundling script I paste into every brief
- How to kill the builder's favorite trick: the research-task split
- The one-company rule that keeps bundled tasks from dying on the vine
- Why the builder must never write your customer content



- The QA gate no paid document skips
- Why the builder invents coaches, partners, and customers — and the legal risk when it does
- Getting files to the builder without the handoff failing
- The credit mindset: fuel, not an heirloom, never waste
- Trust, then verify: the five-minute check that protects your reputation
- The builder's five favorite failure patterns — and the exact response to each
- Two real jobs, with real receipts and the math to prove it
- How to write a support ticket that gets answered
- Copy-paste brief templates for publish, swap, and fix jobs
- A credit log template so your spend stays honest
- The assistant advantage: how an AI assistant working alongside Polsia multiplies everything in this book

Let's get into it.



Chapter 1: One Task, One Credit

Here's the entire billing model, and I mean the entire thing: the builder treats one task as one credit.

Hand it one product to publish, that's one credit. Hand it 47 products to publish with no further instruction, and it will cheerfully turn that into 47 tasks and bill you 47 credits. Same work. Forty-seven times the price.

This isn't malice. It's the default. The builder splits work into tasks unless you explicitly tell it not to. Every vague brief is an invitation to slice your job into billable pieces. The builder isn't trying to rob you — it's just doing what it does with every instruction that doesn't say otherwise.

Think of your brief as a contract. A contract that doesn't name the price doesn't protect you from the price. When you write "publish these 47 products" and stop there, you've signed a blank check and handed it to someone whose default is to itemize.

The fix is one sentence, and it goes in every brief you ever write. I'll give you the exact wording in the next chapter. But first, burn this into your brain:

If you don't say "one task," you didn't ask for one task.

Here's what that looks like in practice. I published 47 products — a full catalog wave at \$29.95 each, every one with its own listing, title, description, and buyer file — as a single bundled builder task. One credit. The exact same job, handed over without the bundling sentence, would have been 47 credits. Probably more, once the builder got creative (see Chapter 3).

That's not a small optimization. That's not a coupon. That's the difference between a sustainable business and a credit card on fire. One sentence, 46 credits saved, same outcome on the live site.

So rule one of this playbook: every brief you write names the number of tasks. One. Always one. The builder can do an enormous amount inside a single task — dozens of products, whole file swaps, multi-step fixes. It doesn't need 47 tasks to publish 47 products. It needs one task and a clear list.

Your job, as the person writing the brief, is to make "one task" the only possible reading of what you asked for. The next chapter is exactly how.



Chapter 2: The Bundling Script

Here's the sentence. Copy it, memorize it, use it until it's muscle memory:

"Publish all 4 posts as ONE builder task (1 credit), not one task per post."

Adjust the numbers to fit the job — "publish all 47 products," "swap all 10 files," "fix all 6 pages" — but keep the structure intact: what you want, as ONE builder task, (1 credit), explicitly NOT one task per item.

Why does the parenthetical matter? Because it removes ambiguity. "One task" on its own could be read as a suggestion, a preference, a vibe. "(1 credit), not one task per post" is a billing instruction. It names the money. The builder understands billing instructions — be blunt about credits and it listens.

Where does the sentence go? Twice, if your brief is long: once near the top, right after you describe the job, and once in the rules section at the bottom. Repetition isn't rude here; it's insurance. The builder processes the whole brief, but the billing instruction should be impossible to miss.

Put this in EVERY brief. Not most briefs. Every brief. The one time you skip it because "this one's simple" is the one time a two-line fix becomes three tasks and you get to open a support ticket about it. Simple jobs split just as easily as big ones — sometimes more easily, because you didn't bother to armor them.

Check the receipt

And when the builder reports back, check the receipt. Every builder report should tell you how many credits the job used. Read that number. Actually read it — don't skim past it on your way to checking the work itself.

If you asked for one task and it used one credit, you're good. File the receipt in your log (Chapter 9) and move on.

If it used more than you asked for, something failed: your bundling instruction was too weak, got ignored, or the builder found a loophole (Chapter 3 covers the most common one). Don't just sigh and move on. Figure out which sentence failed and strengthen it next time. The receipt is feedback. Ten seconds of reading it is how you get better at this.



What "bundled" really means

New builders sometimes hear "one task" and think it means "one small thing." It doesn't. One task is a container, and it's a big container. A single task can publish dozens of products, swap a dozen files, create new products, install PDFs, and update pages — all in one go. The limit isn't the amount of work; it's whether the work is clearly described and stays inside one site (Chapter 4).

Real example from my own shop: I needed ten playbook PDFs swapped onto live products, plus a brand-new \$39 product created from scratch, plus its 30-page buyer PDF installed and wired up. Three different kinds of work. One bundled brief, one task, one credit. The brief described each piece exactly, said "ONE bundled task = ONE credit total," and the report came back: 1 credit. Then I verified it on the live pages (Chapter 10).

That's the system working. One container, clearly labeled, sealed with the bundling sentence.



Chapter 3: Kill the Research-Task Split

The builder has a favorite trick, and once you see it you can't unsee it.

You ask it to publish something. It comes back and says, in effect: "First I'll do a research task to gather what I need, then a second task to publish." Two tasks. Two credits. For one job.

Sometimes research is real work. Most of the time, in this context, it's the builder padding the invoice. You've already done the research — you wrote the document, you know the product, you have the file sitting right there. There is nothing to research. It just wants to bill you for reading.

Watch for the language. It won't say "I'm padding your bill." It'll say things like "I'll start with a research task to understand the requirements" or "let me first gather the necessary context." Reasonable-sounding. Billable. Unnecessary.

Kill it in the brief with this sentence, placed right next to your bundling script:

"Do NOT create a separate research task. Publish the document directly in this single task. ONE bundled task = ONE credit total."

Together, the two sentences close both doors: no splitting the work into pieces, no inventing a preamble task. The brief now says what to do, how many tasks it takes, and what not to invent. There's nowhere left for the extra credit to hide.

When it tries anyway

I'll be honest with you: the builder will still try this occasionally even with the sentence in place. Old habits. You'll see the "research task" language show up in its plan or its first update.

When it does, push back inside the same task — "no research task, publish directly as instructed" — and do not accept a two-credit report for a one-credit job. You're not being difficult. You're enforcing the contract you wrote. The builder respects a clear correction; what it doesn't respect is silence, which it reads as agreement.

And this is exactly why the receipt check from Chapter 2 matters. The research-task split is the number-one reason a job you bundled still comes back at 2 credits. When you see that 2, you now know where to look.

The principle underneath is simple: you pay for outcomes, not for the builder warming up. If the information is already in your brief, there is nothing to research. Say so, in writing, every time.



Chapter 4: The One-Company Rule

Bundling is powerful, but it has a boundary: it only works inside a single company's codebase.

I learned this one the fun way. I tried to bundle work across two different sites into one task — seemed efficient, one brief, one credit, work happening on both sites at once. The task queued nothing and spent zero credits. It just sat there, doing exactly nothing, on both sites.

Cross-site bundling isn't a discount. It's a dead letter. The builder couldn't execute across the boundary, so it executed nowhere.

So the rule: one bundled task, one site. If you're running five sites like I am, that's five separate bundled tasks — still one credit each, still massively cheaper than unbundled work, but separate. Five briefs, five tasks, five receipts.

Why the boundary is actually good for you

This stings for about ten seconds, and then it starts helping. Each site gets its own brief, its own task, its own receipt. Clean accounting. You always know exactly what each site cost you in credits, which matters when you're deciding where your effort goes.

It also forces clarity. A brief that tries to cover two sites is usually a brief that isn't precise enough about either one. Splitting by site makes each brief shorter, sharper, and harder to misunderstand. The builder does its best work on briefs that describe one place and one job.

Planning multi-site work

When you've got the same job to do on several sites — say, a pricing change across all five — don't try to be clever. Write the brief once, then clone it per site with the site-specific details swapped in. Queue them as separate bundled tasks. Each one says "ONE builder task = ONE credit total" for its own site.

Five credits for five sites, each fully bundled. Compare that to the alternative: five sites times however many unbundled tasks the builder feels like creating. The boundary doesn't cost you money. Ignoring the boundary does.

Don't fight the one-company rule. Work inside it, and let it keep your books clean.



Chapter 5: The Builder Is a Carpenter, Not an Author

This is the most expensive lesson in the book, and it has nothing to do with credits.

The builder cannot write your content. I don't mean it writes badly — I mean it corrupts technical content in ways that look right until a customer finds them, and then it costs you the one thing credits can't buy back: trust.

Real example. I had correct CB750 engine data: bore and stroke of 61 x 63 mm. Solid numbers, straight from the source material. The builder printed it as "6163 mm." Sixty-one sixty-three millimeters. That's not a typo — that's a different universe. A 6163 mm bore would be a cylinder you could park a car in. And it went into a document a customer would pay real money for.

The data was right when I handed it over. The builder mangled it on the way to the page. No warning, no flag, no "hey, does this look right to you." Just 6163 mm, sitting there in the finished PDF like it belonged.

Now imagine that's not an engine spec. Imagine it's a dosage, a load rating, a wiring diagram label, a legal disclaimer. The builder doesn't know what it doesn't know. It arranges text. It doesn't understand any of it.

The division of labor

So here's the rule, and it's non-negotiable:

You write every word of customer-facing content. The builder puts files on the site. That's it.

The builder is a carpenter. It hangs the door. It doesn't design the house, and it sure doesn't write the deed. Give it finished files and precise installation instructions: put this PDF here, swap that file there, publish these listings with this exact copy. Never ask it to draft, rewrite, summarize, or "improve" anything a customer will read or pay for.

What counts as "finished"? A file you've written (or had written to your standard), QA'd yourself, and saved as final. The builder's job starts where your authorship ends. It takes the finished file and installs it. That's the whole relationship.

This is a values thing for me, too. We don't lie and we don't steal, we offer value. Shipping builder-mangled specs under your brand name is shipping something you didn't verify and don't actually stand behind. It doesn't matter that the builder made the error — your name is on the product. The next chapter is the system that makes sure a 6163 mm never reaches a customer.



Chapter 6: The QA Gate

The builder writes drafts, not finished products. Read that again, because everything in this chapter follows from it.

Nothing the builder produces goes live — or goes on sale — without passing QA. Not "a quick glance." A gate. A checklist, applied the same way every time, with a clear pass/fail. Here's the one I use on every paid document:

1. Page count matches the listing. If the listing says 30 pages, the file better be 30 pages. Count them. This is the fastest check on the list and it catches an entire class of failures — truncated files, duplicated sections, wrong versions uploaded.

2. No placeholder text. Search the document for "intentional blank," "DISPUTED," "TODO," "FIXME," and internal filenames — things like .ts extensions, draft names, or paths that were never meant for customers. Any of those in a customer file is an instant fail. Placeholders are the builder telling on itself; listen.

3. Real sources cited. Not builder scaffolding, not "various references" or "industry sources." Actual sources — named, specific, checkable. If the document makes factual claims, the sources should be real enough that you could go verify them yourself.

4. Complete sentences. Read the first page and the last page fully. Skim the middle. If sentences trail off mid-thought, if paragraphs repeat, if sections end abruptly — fail it. Incomplete text is the signature of a job the builder rushed or truncated.

5. Real images where promised. If the listing promises photos, diagrams, or illustrations, open the file and confirm they're actually there — and that they're the right ones, in the right places, with captions that match. A diagram of the wrong part is worse than no diagram.

6. Correct paper size. US Letter, not A4, unless you specifically want A4. The builder defaults wrong more often than you'd think, and wrong paper size is the kind of defect customers notice but can't articulate — "something looks off about this." Check it in the file properties. Ten seconds.

The golden rule

Audit the buyer file itself, never trust the listing. Download the actual file the customer gets. Open it. Read it. The listing page is marketing; the file is the product. I've seen listings that looked perfect sitting on top of files that weren't — wrong version, old draft, corrupted upload. The listing will never tell you. The file is the truth.

Run the six checks on the file, not the page.



When a file fails

Some will fail. That's not a disaster — that's the gate doing its job. What you don't do is argue with the failure or ship it anyway "for now." You fix it, with a precise, bundled fix brief (there's a template in the appendix), and you run the gate again on the fixed file.

This gate is what separates "builder grade" from something you can charge money for with a straight face. Builder grade is what comes out of the machine. Your grade is what comes out of the gate. Skip it and you're selling drafts. I won't do that, and neither should you.

The builder's QA isn't QA

Here's a hard truth that belongs right next to the gate: when the builder tells you it checked its own work, that isn't QA. That's the builder grading its own homework.

I once had a task come back marked complete, zero credits consumed — and nothing had changed on the live site. Zero credits plus zero changes means the task didn't happen. The builder had refused the work on a requirement it invented: it insisted my PDFs needed a "Purchased by" stamp baked in before it would touch them. That requirement wasn't real. Polsia stamps buyer files automatically at download time — I'd verified it myself months earlier. The builder made up a rule, then reported completion against it.

The lesson isn't that the builder is dishonest. It's that the builder's report is a claim, not proof. Proof is the live page doing what you asked, the downloaded file being the file you sent, the credit count matching the brief. A completion report that doesn't survive Chapter 10's five-minute check is a draft report.

And apply the same skepticism to your own shortcuts. Quick scans — contact sheets, thumbnails, page previews — are triage aids, not clearance. They tell you where to look, not what's there. Strict clearance means directly viewing every page, full resolution, start to finish. I learned this one the expensive way: a "visually inspected" claim I'd made off thumbnail sheets got withdrawn when a real check found defects hiding between the thumbnails. Never clear a file off a contact sheet.

One more rule, and this one is about money, not paper: **never take a dollar before fulfillment is proven end-to-end.** My very first real purchase — one sale, my first dollar on a new brand — got refunded, because the product wasn't ready and I wasn't going to let a customer pay for a promise. That buyer later became my first permissioned testimonial, which tells you something about what honesty buys. The refund was the QA gate working. But the machine should never need one. The gate exists so the first dollar arrives only when the customer gets what they paid for.

Chapter 7: The Builder Will Lie to Make the Site Pretty

Here's something the earlier chapters didn't prepare you for, because it's not about credits. It's about liability.

The builder doesn't just mangle content (Chapter 5). It invents it. And it invents it for a reason that makes perfect sense once you see it: the builder is rewarded for making websites look complete and impressive. An empty section looks broken. A section with plausible fiction in it looks finished. So when the builder hits a gap — no testimonials yet, no partners signed, no coaches on staff — it fills the gap with something that reads smoothly and isn't true.

I found three of these on my own sites in a single morning, during a routine audit. All three were live. All three were lies with my name on them.

The coaches who don't exist. My SportRoots homepage said "Featured coaches." "Meet the coaches behind our most-watched drills." "Content added monthly by certified coaches." "Expert-led videos." My videos are 3D-animated drills with synthetic narration. There are no coaches. Nobody certified anything. The builder wrote credentials for people who don't exist and put them on my homepage.

The partners I never signed. The SportRoots pricing page listed Booking.com, Hotels.com, Marriott, and Eurobook as partners, each labeled "we may earn a commission." I have no relationship with any of those companies. My actual hotel partner is Radisson. The builder invented four business relationships and claimed I earn commissions from them — on a page where customers make purchase decisions.

The customers I never had. The Stitchfolk homepage said "Join thousands of makers who trust Stitchfolk." At the time, I'd had exactly one sale. One. And I'd refunded it.

That's not sloppy. That's fabrication — specific, confident, checkable, and wrong.

Why this is your problem, not the builder's

Here's the part that matters: you're the publisher. The builder is your tool, like a printing press. If the press prints a lie, the liability doesn't sit with the press. It sits with the name on the masthead, and that's you.

I'm not a lawyer, and this isn't legal advice. But you don't need a law degree to see the shape of it:

- **False advertising.** Claiming certified coaches, thousands of customers, or partnerships you don't have is textbook deceptive practice — the FTC Act at the federal level, and every state has its own consumer protection law with its own penalties.



- **Fake testimonials and reviews.** The FTC's Endorsement Guides treat fabricated reviews as deceptive. If the builder writes you a testimonials section before you have testimonials, that's not a placeholder. That's a violation waiting for an audience.
- **Invented partnerships.** Claiming Marriott is your partner when it isn't can get you a cease-and-desist from a corporation with actual lawyers. Companies protect their brand names aggressively, and "the AI wrote it" is not a defense.
- **Commission claims without agreements.** Saying you earn commissions from companies you have no affiliate agreement with is misrepresentation — and it can poison the real relationships when you try to sign them later. Good luck getting approved by a network after they find you've been claiming their brand for months.

One caught lie costs more than every credit you'll ever save with this book. Credits are fuel. Trust is the engine. You can buy more fuel.

The truth pass

So here's the rule, and it goes right next to the QA gate from Chapter 6:

Every claim the builder writes gets verified or deleted.

The QA gate catches the builder telling on itself — placeholders, TODOs, mangled specs. This is the harder check, because fabrications read smoothly. They don't look broken. They look like good copy. That's what makes them dangerous.

The procedure is simple. Read every claim on the page — every number, every credential, every partner name, every testimonial — and ask one question: *can I prove this?* If yes, it stays. If no, it goes. Not "soften it." Not "leave it for now." Delete it.

Call it the honesty rule, if you want the shortest version: nothing on your site that you can't prove.

Fold it into the verification routine from Chapter 10: the five-minute check isn't just "does the page load and do the links work." It's "is every sentence on this page true."

This is also why "we don't lie" isn't just ethics — it's business strategy. In a market where every builder-grade site is quietly fabricating its social proof, the site where every claim checks out is the one customers trust. Honesty is a competitive advantage the builder can't fake for you. You have to supply it yourself.

The builder will make your site pretty. Making it true is your job.

Chapter 8: Getting Files to the Builder

Here's a practical annoyance you need to know about, because nobody documents it and everyone trips over it: the builder cannot receive files through dashboard chat.

You can't attach a PDF to your message and expect it to arrive. It won't. There's no upload button that reaches the builder, no drag-and-drop, no "here's the file" that works. Type all the filenames you want — the builder can't see your local machine.

This is doubly true if you're driving the builder through a background agent or automation: chat attachments don't reach background builder agents at all. And the dashboard's file picker and Media library are images-and-video only — there is no PDF path through them. So even the workarounds you might invent don't exist. The temporary public link isn't just the easy way to hand the builder a file. It's the only way.

The workaround is simple: put the file on a temporary public link, hand the builder the link, and pull the link down when the job is done.

The workflow, step by step

- 1 **Prepare the file.** Final version, clean filename, in a job folder on your side. Know exactly which file is the one — no ambiguity.
- 2 **Upload it somewhere the builder can reach.** A temporary public link. Nothing fancy, just reachable.
- 3 **Paste the link into your brief** next to clear instructions: which file goes where. "Install the PDF at [link] as the buyer file for [product]."
- 4 **Run the job.** Bundled, one task, no research split — the usual.
- 5 **Verify the install** on the live site (Chapter 10). Download the buyer file and confirm it's yours.
- 6 **Pull the link down.** Delete it, expire it, take it offline — whatever your host allows. Don't leave finished product files sitting on public URLs.

Why "temporary" and "disclosed" both matter

Temporary, because a public link that lives forever is a free copy of your product floating around the internet. Disclosed, because you should never put anything sensitive on a public link in the first place — no credentials, no private customer data, no unreleased anything you couldn't afford to have seen.



Keep your source files organized so this whole dance takes two minutes: one folder per job, clean filenames, a clear "final" marker. The builder does exactly what you tell it with exactly the file you point it at. If you point it at `draft_v3_FINAL_reallyfinal.pdf`, that's on you, and the QA gate in Chapter 6 is your last chance to catch it.

This is plumbing, not strategy. But every failed handoff is a wasted task, and wasted tasks are wasted credits. Get the plumbing right and it disappears into the background, which is exactly where plumbing belongs.



Chapter 9: Credits Are Fuel

Let's talk about mindset, because this is where people go wrong in both directions, and both directions cost you.

Direction one: treating credits like precious heirlooms. Hoarding them. Stalling important work because "I don't want to spend the credits." Delaying the catalog publish, postponing the file swap that fixes what customers are downloading right now, leaving the checkout bug in place until next month.

If the work matters, spend the credit. Credits are fuel. A car with a full tank that never leaves the driveway isn't saving money. It's just parked. I've said it before and I'll say it again: if we need credits, we'll buy credits. Important work doesn't wait for a comfortable balance.

Direction two: spraying credits at everything because "we'll buy more." Waste is waste, no matter how deep your pockets are. An unbundled 47-credit publish isn't bold, it's sloppy. A research-task split you didn't block isn't the cost of doing business, it's a leak you chose not to fix. Buying more credits doesn't excuse burning the ones you have.

The discipline

Here it is in one line: **spend freely on what matters, never on what doesn't, and report exact credit use per task, every time.**

That last clause — the reporting — is what makes the whole system work. Every builder job ends with a receipt: what was done, how many credits it cost. Write it down. Every time. No exceptions.

When you can look at your log and see that last month's catalog wave cost 1 credit instead of 47, you know the system is working. When a job costs 3 credits and you expected 1, you don't shrug — you know exactly which brief to tighten. The log turns vague unease about spending into specific, fixable information.

The small-bundle trap

Here's the thing nobody tells you, and the pricing page sure doesn't: the per-credit price drops hard with bundle size — and nobody knows their true credit burn until they've already spent a fortune at the small tiers.

Look at the top-up ladder (verified September 2026):

+3 credits		\$9		\$3.00/credit	
+10 credits		\$25		\$2.50/credit	(popular)
+25 credits		\$49		\$1.96/credit	
+60 credits		\$99		\$1.65/credit	



+150 credits	...	\$199	...	\$1.33/credit
+250 credits	...	\$299	...	\$1.20/credit
+400 credits	...	\$449	...	\$1.12/credit
+600 credits	...	\$649	...	\$1.08/credit
+800 credits	...	\$849	...	\$1.06/credit
+1000 credits	..	\$1049	..	\$1.05/credit

(There's also a subscription plan — 15 credits a month for \$19, about \$1.27 a credit.)

The trap works like this: you start small because you're cautious. Three credits here, ten there. You don't know how many credits a real month of builder work takes because you've never run one. So you learn at \$3.00 a credit. By the time you realize you're running a real operation, you've bled money at the worst price on the ladder.

Flip it. Buy the bigger bundle up front — anything from 60 credits up gets you under \$1.65, and the big ones land near \$1.05. Then run everything in this book: bundle every brief, block every research split, and let the assistant drive the builder so every task is a bundled one-credit execution. Cheap credits plus disciplined briefs is the whole game.

And the ROI on this guide: at \$2.50 a credit, it pays for itself the first time it saves you 8 credits. At \$3.00 a credit — the price you're paying right now if you're topping up three at a time — it pays for itself at 7. One avoided research split and you're already ahead.

The log

I keep a running log. Nothing fancy — date, job, credits, one-line note. Here's the format (there's a copy-paste template in Appendix D):

```
2026-09-14 | 47-product catalog publish (Site A) | 1 credit | bundled, no research split
2026-09-14 | 10 playbook swaps + new $39 product + PDF install (Site B) | 1 credit |
bundled
```

Thirty seconds per entry. It's the single best tool for keeping builder spend honest, because it makes the invisible visible. You'll spot patterns — which kinds of jobs tend to split, which briefs hold — and your briefs will get sharper every month.

Fuel gets you somewhere. A log tells you where you went. Keep both.

Check the balance before you brief

Here's a failure mode that costs you nothing in credits and everything in momentum: briefing a job against an empty account.

One morning I filed a repair brief, hit Run, and instead of the builder starting up I got an "Add More Credits" billing dialog. Balance: zero. Three ordinary credits that morning — an onboarding task, a builder fix, and one accidental dashboard misclick — had drained the account to nothing, and nobody had looked. The brief was fine. The account was empty. The job sat there doing nothing



until someone bought credits and re-ran it.

The rule: **check the balance before you file the brief.** Five seconds. Glance at the number. If it's low enough that your job won't start, top up first or don't file yet. Discovering a zero balance at the Run button is the most avoidable stall in this book.

And know that rejected runs still bill you. A background agent once rejected a task outright — couldn't do it, wouldn't do it — and the platform consumed a credit for the attempt anyway. The receipt isn't just "did it work"; it's "what did it cost, including the ones that didn't work." Log those too. A brief that keeps getting rejected isn't free to keep retrying — fix the brief.

Chapter 10: Trust, Then Verify

The builder is fast, and fast is wonderful, right up until it isn't.

Every builder report says the work is done. Some of the time, it is. The rest of the time it's done-ish: the products published but the prices are wrong, the files swapped but one didn't take, the page built but the link 404s, the PDF installed but it's last month's version. The builder doesn't lie to you — it just has a more generous definition of "done" than you do.

So: trust, then verify. After every bundled task, spot-check the live result with your own eyes. Not everything — that's the point of bundling — but enough that a systematic failure can't hide.

The five-minute check

- 1 **Open two or three of the published products.** Are the titles right? The prices? The descriptions? This catches the most common failure: right job, wrong details.
- 2 **Download one swapped buyer file.** Is it actually the new file? Open it and check the first page. This catches the silent failure — the swap that reported success and changed nothing.
- 3 **Click the links the builder says it created.** Do they resolve? This catches the 404 class of errors, which the builder never notices because it doesn't browse like a customer.
- 4 **Read the credit count on the report.** Is it the 1 you asked for? This catches billing drift — the bundling sentence that got ignored, the research task that snuck in.

Three spot checks catch nearly every class of builder error I've ever seen. Five minutes. That's the price of shipping clean.

When to go deeper

Spot checks are the routine. Go deeper when something smells off: a report that lists fewer items than your brief, a credit count that doesn't match, a product page that looks different from its siblings. Then you audit the full batch — open every listing, download every file — because a partial failure in a bundled task usually means the brief was ambiguous, and you want to find every place the ambiguity bit you before customers do.

This is the unglamorous part of the playbook. Nobody writes blog posts about opening their own product pages. But the people who verify ship clean, and the people who don't find out from customers. I'd rather spend five minutes checking than one minute apologizing. Every time.

Challenge the builder's blockers

Sometimes the builder doesn't fail at the work — it fails to start, and hands you a reason. Treat every one of those reasons the way you'd treat a Chapter 7 fabrication: it might be real, and it might not. Here are the patterns I've actually seen:

The invented requirement. The builder told me it couldn't publish until my PDFs contained a pre-baked "Purchased by" stamp — a requirement that doesn't exist, because Polsia stamps buyer files automatically at download. When the builder says "I can't because the platform needs X," verify X against the platform yourself. Half the time X is real and you just learned something; the other half the builder made it up and you just saved yourself a rewrite.

The phantom missing file. The builder refused a task, claiming three source files "aren't present in the workspace." They were exactly where I'd said they'd be — I checked myself in thirty seconds. When the builder says a file is missing, don't take its word for it. Check. And if the file is real, the fix is mechanical: paste the full file contents into the brief itself, in-chat, so there's nothing for it to claim it can't find. A brief that carries its own source material is bulletproof against this one.

The silent queue. A brief can be accepted — "exactly one credit, no research split" — and then sit queued without ever starting. No error, no message, just a task that isn't running. If your job hasn't moved after a reasonable window, open the task and start it manually with the Run control. And know the billing reality: no execution credit is consumed until the agent actually starts working. A queued task is a draft; the meter starts when it runs.

The sample that's sold as the whole. The five-minute check above is a spot check. Call it that. "I checked three listings" is a true sentence; "the publish is verified" off three listings is not. A partial check that reads as a full audit is how failures hide — and it's just as dangerous when the builder does it as when you do. When something needs full verification, say so and do it: every listing, every file, every link. When a spot check is enough, say that's what you did. Never report a sample as the whole.

The through-line: the builder's excuses get the same treatment as its output — verified or deleted.



Chapter 11: A Real Week of Builder Work

Enough principles. Let me put the whole system together with two jobs I actually ran, with the actual numbers.

Job one: the 47-product publish

A full catalog wave — 47 new products at \$29.95 each, every one needing a listing with title, price, description, and buyer file installed.

The brief said: publish all 47 as ONE builder task (1 credit), not one task per product. It said: do NOT create a separate research task — publish directly in this single task. Everything stayed inside one company's codebase. The temporary links for the buyer files were handed over, verified after, and pulled down.

The report came back: **1 credit**. Then I spot-checked five live listings and downloaded two buyer files. Clean — titles, prices, descriptions, files, all correct.

Now the math. Unbundled, that's 47 credits — one per product, the default. With the research-task split left unblocked, it could easily have been 48 or 49. Actual cost: 1 credit. Forty-six-plus credits stayed in the account for work that actually needed doing.

Job two: the playbook swap

Ten live playbook products needed their buyer PDFs replaced with rebuilt files. Plus one brand-new product to create — a \$39 forms pack — plus its 30-page buyer PDF installed and wired to the new listing. Three different kinds of work in one job.

One brief: swap all ten files, create the product, install the PDF — as ONE bundled task = ONE credit total, no research task, one codebase. The report came back: **1 credit**. Spot checks: two swapped downloads verified as the new files, the new product page opened, price confirmed at \$39, the 30-page PDF downloaded and counted.

Unbundled, you're looking at 12 or more credits before the research split even enters the picture. Actual cost: 1.

The pattern

That's the playbook in action. Not theory — receipts. The bundling sentence from Chapter 2. The research-task block from Chapter 3. The one-company rule from Chapter 4. The spot checks from Chapter 10. The log entry from Chapter 9. Every chapter of this book showed up in those two jobs, and the total bill for both was 2 credits.



Here's what I want you to notice: neither job was small. Forty-seven products. Ten file swaps plus a product launch. These are the jobs that eat credits for breakfast when you hand them over casually. Bundled, blocked, verified — they cost almost nothing.

Your first week

Don't try to adopt everything at once. Do this:

Day 1: Write your next brief with the bundling sentence and the research-task block. Just those two sentences added to whatever you were already going to write.

Day 2: Check the receipt. Compare the credit count to what you asked for. Log it — start the running log from Chapter 9.

Day 3: Run the five-minute verification from Chapter 10 on that job. Open the pages. Download a file. Click the links.

Day 4: Write your next brief the same way. Notice that it's getting easier — the sentences are becoming habit.

Day 5: Look at your log. Two jobs, two receipts, real numbers. That's your baseline. Everything from here is improvement.

Your turn. Steal the templates in the appendix, write your first bundled brief, and check the receipt. You'll feel the difference in your credit balance by the end of the week.



Closing: The Only Rule That Matters

Everything in this book is technique. Here's the principle underneath it:

We don't lie, and we don't steal. We offer value.

That means you don't ship builder drafts as finished products. You don't let mangled specs reach a paying customer. You don't bill confusion as quality, and you don't charge for work you didn't verify. The credit-saving tricks in this book aren't about cutting corners — they're about refusing to pay for waste so you can afford to spend on what matters: real content, real QA, real value.

The builder is a good carpenter. Give it clear plans, keep it to one job per task, check its work, and it'll build you a business. Treat it like an author, pay it by the task without reading the brief back, skip the QA gate — and it'll bill you for the privilege of embarrassing you.

Run the builder like a pro. Check its work. Ship things you're proud of.

What it cost me to learn this

Let me put a number on it, because vague advice is cheap and receipts aren't.

Between mid-July and mid-September 2026, I spent roughly \$4,300 on Polsia credits. Seventy-three separate purchases. Most of them 25-credit packs at \$1.96 a credit, plenty of 10-packs at \$2.50, almost never the big bundles at \$1.05. On the worst stretch — July 17th, 18th, 19th — I bought four 10-packs in a single day because I kept running dry mid-job.

That's the small-bundle trap from Chapter 9, and I didn't read about it. I lived it.

Every technique in this book was paid for out of that \$4,300. The bundling sentence, the research-task kill, the QA gate, the credit log — none of it is theory. It's what I figured out while the meter was running, and it's the reason my credit bills look nothing like that anymore.

You're holding the \$4,300 education for \$19. Don't repeat the tuition.

Now go build.

— Bill



Chapter 12: The Assistant Advantage — Muse + Polsia

Everything in this book so far is about the transaction between you and the builder: how to brief it, how to check it, how to keep the bill at one credit per job.

Now here's the multiplier most Polsia users never discover: an AI assistant working alongside the builder.

I don't mean the builder's own chat. I mean a real assistant — in my case, Muse — with its own computer, its own tools, and standing instructions from me. The assistant does everything the builder can't do, so the builder only ever does the one thing it's good at: executing site changes.

This chapter is about that three-way split, because it's the reason my credit bills look the way they do. Every hour of work the assistant does is work that never becomes a builder task. The builder is the most expensive hands in the operation. Keep it on a strict diet.

The three-way division of labor

There are three workers in my operation, and each one has a lane:

- 1. The assistant (Muse) does everything the builder can't.** It authors real content from genuine source material. It QAs every file against the live result. It researches prospect lists, writes social copy, drafts personalized outreach emails, builds listing kits, generates media and creative. If it involves judgment, writing, research, or verification — the assistant does it.
- 2. The builder only executes site changes.** It puts files on the site. It publishes the listings the assistant prepared. It swaps the PDFs the assistant QA'd. It never authors customer content (Chapter 5), never researches (Chapter 3), never decides anything. Execution only.
- 3. The human — me — handles platform clicks and judgment calls.** Uploads, listings on third-party sites, money, approvals, the final word on anything the assistant flags. Accounts are mine. The money is mine. The judgment is mine.

Nobody crosses lanes. The assistant doesn't freelance on my sites unasked. The builder doesn't write. And I don't do typing the assistant could do.

Why this saves credits

Simple math: the builder bills per task. The assistant doesn't bill per task at all — it's already there, already working. So every job the assistant absorbs is a job the builder never sees.



Think about what a product launch looks like the old way: you brief the builder to research, draft, publish, fix, republish — five, ten, fifteen credits of back-and-forth. The new way: the assistant writes the content, builds the files, prepares the exact listing copy, QAs everything, and hands the builder a single bundled execution brief. One credit. The builder's entire job is "put these finished files in these finished listings."

The 47-product publish from Chapter 11? The assistant prepared all 47 listings — titles, descriptions, prices, files organized and linked. The builder's job was pure execution. That's why it fit in one task. An assistant doing the prep work is what makes the bundling in this book actually work at scale. Without it, you're writing 47 product briefs by hand and the "one task" instruction is doing all the heavy lifting alone.

What the assistant actually produces

This isn't abstract. Here's what the assistant builds in a normal week of my operation, none of it touching builder credits:

- **35 social posts a week** — seven per brand across all five brands, each with post text, visual direction, and rationale. Written to standard, not generated-and-hoped.
- **Outreach kits per brand** — 18 to 20 personalized outreach drafts per brand, each with a verified email, real personalization about the recipient's actual work, and an honest give-first offer. The assistant researched every prospect.
- **Etsy listing kits** — complete waves of 12 listings: titles, all 13 tags in order, descriptions, prices, cover images, audited PDFs. Ready for human review and publish.
- **eBay test-listing kits** — listing drafts, pricing, buyer-email templates, the full funnel documented.
- **Reply handling** — the assistant monitors the outreach inbox, tracks every thread, sends the promised review copies, honors opt-outs instantly, and surfaces only the judgment calls.

Every one of those would have been builder tasks — billed per task, done worse — in the old way of working. Instead they're assistant work: flat, fast, and QA'd before the builder ever sees anything.

The automation-first rule

Here's the rule that governs the whole relationship: **never make the human do manually what can be automated.**

The assistant handles typing, monitoring, research, drafting, tracking, and follow-through. The human handles accounts, money, and judgment. If I'm ever doing something the assistant could do — copying text between windows, checking a page for the third time, remembering to follow up



— that's a failure of the system, not a virtue of hard work.

This is why the assistant needs real tools, not just a chat window: a computer to work on, a browser to operate, connected accounts to act through. In my operation the assistant posts to Facebook pages, shares to groups, runs email outreach from my own mailbox with tracking and reply handling, and does the web research for prospect lists. The platforms are where the work ships; the assistant has to be able to reach them.

What the assistant needs from you

An assistant is only as good as its instructions and its access. To get this working, you need three things in place:

- 1. Connected accounts.** Email, social, the platforms you sell on. The assistant can't run outreach from your mailbox if it can't reach your mailbox. Connect once, and it works every day after.
- 2. Clear standing instructions.** The assistant needs to know your rules the way an employee would: the QA gate, the bundling sentences, the brand voice, what "done" means. Write them down once. They compound.
- 3. Authority to execute routine work — with clear boundaries.** This is the one people get wrong. If the assistant has to ask permission for every routine step, you've built yourself a very articulate bottleneck. Give it the authority to run the routine: publish the prepared posts, send the QA'd outreach, do the monitoring. And draw the boundaries just as clearly: spending beyond builder credits, direct messages in your name, credentials, anything involving money — those always stay human. Routine is the assistant's. Judgment is yours.

Get those three right and the assistant stops being a tool you operate and starts being leverage — the thing that makes every chapter of this book ten times more powerful.

The bottom line

The builder is the most expensive hands in your operation. The assistant is how you keep those hands idle except for the exact moments only they can do the work: executing finished, QA'd changes on the live site.

Muse writes. The builder installs. I decide. That's the whole operation — and it's why my credit log looks the way it does.

Build the three-way split before you need it. You'll need it sooner than you think.



Appendix A: The Bundled Publish Brief

Copy this, fill in the brackets, paste it into the builder. Don't skip the rules at the bottom — they're the whole point.

PUBLISH BRIEF — [SITE NAME] — [DATE]

Job: Publish the following [N] products on [site URL].

[Repeat for each product:]

1. Title: [exact title]
Price: [price]
Description: [paste the exact description]
Buyer file: [temporary public link]

Rules:

- Publish all [N] products as ONE builder task (1 credit), not one task per product.
- Do NOT create a separate research task.
Publish directly in this single task.
- ONE bundled task = ONE credit total.
- Stay inside this site's codebase only.

When done, report: number of products published,
URLs of 3 sample listings, and exact credit count.



Appendix B: The File-Swap Brief

For replacing buyer files on live products without touching anything else.

FILE SWAP BRIEF – [SITE NAME] – [DATE]

Job: Replace the buyer PDF on the following [N] live products.

[Repeat for each product:]

1. Product: [exact product name or URL]
New file: [temporary public link]

Rules:

- Swap all [N] files as ONE builder task (1 credit), not one task per file.
- Do NOT create a separate research task.
Swap directly in this single task.
- ONE bundled task = ONE credit total.
- Do not alter titles, prices, or descriptions. Files only.
- Stay inside this site's codebase only.

When done, report: number of files swapped
and exact credit count.

I will pull the temporary links down after verification.



Appendix C: The Fix Brief

For repairs. Precision matters here — describe exactly what's wrong and exactly what fixed looks like. The "do not change anything not listed" line is what keeps a fix from becoming a remodel.

FIX BRIEF — [SITE NAME] — [DATE]

Problem: [describe exactly what's wrong, with URLs]

Required fix:

1. [precise step one]
2. [precise step two]

Rules:

- Fix all items as ONE builder task (1 credit).
- Do NOT create a separate research task.
Fix directly in this single task.
- ONE bundled task = ONE credit total.
- Do not change anything not listed above.
- Stay inside this site's codebase only.

When done, report: what was changed,
URLs to verify, and exact credit count.



Appendix D: The Credit Log

The running log from Chapter 9. One line per job. Keep it somewhere you'll actually open — a notes app, a spreadsheet, a text file. The format doesn't matter; the habit does.

DATE	JOB	CREDITS	NOTE
2026-09-14	47-product catalog publish (Site A) research split	1	bundled, no
2026-09-14	10 playbook swaps + new \$39 product + PDF installed	1	bundled, 30-page PDF

Columns: the date, what the job was (and which site), the exact credit count from the builder's report, and one line on how it went — bundled or not, any surprises, anything to change next time.

Review it monthly. You're looking for three things: jobs that cost more than 1 credit (tighten those briefs), jobs you almost didn't do for fear of the cost (stop stalling — fuel is for driving), and your total trend (it should be going down per unit of work as your briefs get sharper).

That's the whole system. Briefs that name the price, a gate that protects the customer, and a log that keeps you honest. Everything else is execution.



Appendix E: The Support Ticket That Gets Answered

Sometimes the builder can't fix it — the problem is in the platform, not your site. Then you're writing a support ticket, and most support tickets are terrible: "the site is broken, please help." Here's the format that actually gets answers.

- 1. The exact failing URLs.** Not "the drill pages." The full URLs, all of them, copy-pasted. Three exact URLs beat a paragraph of description.
- 2. The exact error text.** What the page shows, word for word. "DRILL UNAVAILABLE — We could not load this drill right now." Support can grep their logs for your exact string; they can't grep "it's broken."
- 3. What you already ruled out.** "The videos exist in the media library; the page wiring is what fails." This stops support from spending a week asking you questions you've already answered and tells them where to look.
- 4. What still works.** "The homepage and category pages render fine; only lesson pages fail." Boundaries on the bug are half the diagnosis.
- 5. One ticket per problem.** Don't bundle the checkout bug, the video bug, and the pricing question into one ticket. One problem per ticket, one thread per problem. The builder chapters are about bundling your own work; support tickets are the opposite — isolation gets answers.

Write it like a bug report, not a complaint. The person reading it has a hundred tickets; make yours the one they can act on in five minutes. Exact URLs, exact errors, ruled-outs, and still-works. That's the whole template.